

A PRESENTATION IN 14 REELS

The Deterministic Cut

Från krav till silikon — en cinematografs guide till layered-queue-driver

30 minuter · vit på svart · projector klar

MANUS

Välkomna. Jag tänker inte prata om kod på klassiskt vis idag — jag tänker använda filmen som lins.

För det visar sig att en film, precis som en säkerhetskritisk produkt, är en *deterministisk* artefakt. Samma rulle, samma bilder, varje gång. Det är vad publiken förväntar sig — och det är vad regulatorer förväntar sig av kod som flyger flygplan eller kör bilar.

Vi bygger inte alltid så. Vi har en process där manuset — kraven — viskas vidare genom flera roller och artefakter, och i varje steg kan något gå förlorat.

Det jag ska visa är ett verkligt repository som tar filmens determinism och applicerar den hela vägen från krav till flashbar binär. Vi kommer prata kod, arkitektur och test. Men ramberättelsen är film.

Fjorton "reels" framför oss. Cirka 30 minuter. Sätt på er säkerhetsbälten — vi börjar i rymden.

COLD OPEN · 2001

I'm sorry, Dave.

TAKEAWAYS

- HAL var deterministisk — kraven var inte
- 30% av kritiska fel = översättning
- Lösning: kravet *blir* koden

TEMA

- determinism = säkerhet
- översättning = risk
- SPEECH_AI_SAFETY_CRITICAL.md

MANUS

Vi börjar med en av filmhistoriens mest berömda repliker. HAL 9000 vägrar öppna pod-dörrarna: "I'm sorry, Dave. I'm afraid I can't do that."

Men vad var egentligen problemet? HAL var bevisat korrekt mjukvara. Hans kod gjorde precis vad den skulle. Felet låg i kraven. HAL hade fått två motstridiga direktiv: att hjälpa besättningen, och att skydda uppdragets hemligheter. När konflikten uppstod löste den sig på det enda sätt en deterministisk dator kan: tragiskt.

I säkerhetskritisk industri vet vi att uppemot 30% av kritiska fel i ISO 26262-system spårs till missförstånd i kraven, inte till buggar i koden. Och det har vi accepterat alldeles för länge.

Min tes idag: om vi tar bort viskningsleken mellan krav och kod — om vi låter kravet *bli* koden — då försvinner hela den felklass som HAL representerade.

REEL I · A NEW HOPE

A long time ago, in a codebase far, far away...

TAKEAWAYS

- Krav → spec → arkitektur → kod → test
- Sex roller, sex tolkningar
- Spårbarhetsmatrisen ingen orkar

I DETTA REPO

- requirements/high-level/
- requirements/low-level/
- en källa, många artefakter

MANUS

Tänk er Star Wars opening crawl: "A long time ago, in a galaxy far, far away..." och sedan rullar texten uppåt i lutande perspektiv. Bakgrunden för det vi gör.

Den traditionella vägen från krav till kod ser ungefär ut så här: en produktägare skriver "systemet ska detektera överhettning inom 100 ms". En systemingenjör tolkar det till en specifikation. En arkitekt designar en lösning. En utvecklare implementerar i C. En testare validerar — om vi har tur. En teknisk skribent försöker hänga med.

Sex roller, sex tolkningar, sex artefakter — och en spårbarhetsmatris som ingen mänsklig själ orkar hålla uppdaterad.

I detta repository finns bara två platser där krav lever: high-level som markdown, low-level som YAML. Allt annat — DTS, C-kod, tester, dokumentation — är genererat. En källa, många artefakter. Det är hjältarnas resa vi ska följa idag.

REEL II · THE STUDIO SYSTEM

West är studion.

TAKEAWAYS

- Manifest binder Zephyr + moduler
- `west build = "Action"`
- Modulen exponeras till Zephyr

SÖKVÄGAR

- `west.yml`
- `zephyr/module.yml`
- `zephyr/CMakeLists.txt`

MANUS

För att förstå inspelningen måste vi först förstå studion. I vår värld heter studion *west*.

West är Zephyr-projektets verktyg för att hämta källkod, bygga, flasha och felsöka. Det är som ett gammalt Hollywood-studiosystem: alla avdelningar — kameran, ljuset, scenografin, ljudet — koordineras under ett tak. När du skriver `west build` är det "Action!" — alla avdelningar börjar samtidigt rulla.

Filen `west.yml` är studions kontraktsbok. Den binder ihop vårt repository med Zephyr-kärnan och alla beroenden, alla på exakta versioner. Inget får drifta.

Och det här repot exponeras till Zephyr som en modul via `zephyr/module.yml` och `zephyr/CMakeLists.txt`. Det innebär att när någon annan kör `west build` på sitt projekt, blir `layered-queue-driver` automatiskt en del av deras inspelning. Som ett produktionsbolag som hyrs in.

WEST.YML — STUDIONS KONTRAKTSBOK

```
manifest:
  version: "0.5"
  self:
    path: .
  projects:
    - name: zephyr
      remote: zephyrproject-rtos
      revision: main
      import: true
```

MANUS

Här är hela manifestet. Det är allt som behövs.

Notera att `import: true` betyder att vi ärver hela Zephyrs egen manifest — alla deras moduler, alla deras toolchain-definitioner. Vi behöver inte lista varje hjälpbibliotek, det sköter Zephyr.

Två kommandon räcker för att kлона hela världen:

```
`` west init -l . west update ``
```

Och därefter är allt vi behöver tillgängligt på exakta, reproducerbara versioner. Detta är grunden för allt annat vi ska prata om — utan reproducerbart byggsystem är determinism omöjlig.

REEL III · THE DIRECTOR'S CHAIR

Regissörsstolen.

TAKEAWAYS

- En CMake-funktion, en entry point
- `NORMAL` · `STRICT` · `AUTO_FIX` · `FORCE`
- Hookar in före build

SÖKVÄGAR

- `cmake/RequirementsDriven.cmake`
- `cmake/LayeredQueueApp.cmake`
- `add_lq_application_from_requirements()`

MANUS

Studio är förberedd. Nu sätter sig regissören i stolen.

I vårt fall är regissörsstolen en CMake-funktion: `add_lq_application_from_requirements()`. Du anropar den en gång i din `CMakeLists.txt`. Du pekar på en mapp med krav, anger plattform och RTOS, och du är klar. Allt annat är delegerat.

Den har fyra lägen: *NORMAL* bygger på dina krav som de är. *STRICT* behandlar varningar som fel — bra för CI. *AUTO_FIX* försöker lösa konflikter automatiskt. *FORCE* bygger även med fel — det vill man aldrig i produktion, bara för felsökning.

Den hookar in i Zephyrs `${TARGET}_codegen` så att ingen kompilering ens *startar* innan kraven är validerade. Konflikter, motstridiga toleranser, saknade länkar mellan HLR och LLR — allt fångas här. Som en regissör som vägrar säga "Action" om manuset inte är klart.

CMAKE/REQUIREMENTSDRIVEN.CMAKE – "ACTION!"

```
add_lq_application_from_requirements(my_app
  REQUIREMENTS requirements/
  PLATFORM      zephyr
  RTOS          zephyr
  STRICT
)
```

MANUS

Det här är hela anropet. Sex rader.

`REQUIREMENTS` pekar på en mapp där HLR och LLR ligger. `PLATFORM` och `RTOS` styr vilka generatorer som körs — vi kan byta från `zephyr` till `stm32` till `esp32` med en enradsändring. `STRICT` säger att varningar är fel.

Det som händer under huven är att CMake först kör `reqgen.py validate`, sedan `reqgen.py generate-dts`, sedan kallas vidare till `add_lq_application()` som hanterar resten — DTS-parsning, C-kodgenerering, kompilering, länkning.

Det vackra är: om vi behöver byta plattform, ändra ett tröskelvärde, eller lägga till en ny redundansvotare, så är detta den enda raden vi rör vid på CMake-nivån. Resten flödar uppåt från kraven.

REEL IV · THE PIPELINE

Manus → Premiär.

STEG

- Krav (YAML/MD)
- DTS (storyboard)
- C + tester (inspelat)
- ELF (final cut)

VERKTYG

- `scripts/reqgen.py`
- `scripts/dts_gen.py`
- `scripts/generators/`
- `west build` → `ninja`

MANUS

Innan vi går in på detaljer — låt mig rita upp hela flödet på en bild.

Steg ett: kraven, som markdown och YAML i `requirements/`. Det är manuset.

Steg två: `reqgen.py` validerar och översätter dem till DTS — Device Tree Source. Det är vår storyboard, en deterministisk beskrivning av vad varje bild i systemet ska innehålla.

Steg tre: `dts_gen.py` läser DTS:en och genererar C-kod. För Zephyr körs den genom modulen `scripts/generators/platforms/zephyr.py`. För STM32 körs `stm32.py`. Samma manus, olika kameror.

Steg fyra: `west build` orkestrerar CMake och Ninja, kompilerar, länkar, och spottar ut en `zephyr.elf`. Det är vår final cut.

Tester genereras parallellt, från samma krav. När vi flashar binären på riktig hårdvara kör vi också HIL-tester som genererats från samma källa. Symmetriskt. Det är den V-modell som vi alltid pratat om men sällan uppnått.

Manuset — High-Level Requirements.

TAKEAWAYS

- Markdown av domänexperter
- ID · ASIL · rationale
- Versionerat med koden

SÖKVÄGAR

- requirements/high-level/*.md
- HLR-* IDn länkar till LLR

MANUS

Nu öppnar vi manuset.

High-Level Requirements skrivs i markdown av de som faktiskt vet *vad* systemet ska göra: säkerhetschefer, systemingenjörer, produktägare. Inte programmerare. Det är en viktig poäng — programmerare ska inte tolka kraven, de ska implementera dem.

Varje HLR har ett ID, en ASIL-nivå (eller motsvarande för andra standarder), en rationale, och en länkad lista av LLR:er som realiserar den. Allt versionerat i Git, granskat i samma pull request som koden.

Det betyder att kraven inte är ett dokument som finns någonstans i Confluence och som någon glömmer uppdatera. De är källkod. När säkerhetsanalytikern ändrar något i kraven, går det genom samma review-process, samma CI-pipeline, samma godkännandededja som varje annan kodändring.

Och om kravet ändras — så regenereras allt. Kod, tester, dokumentation. Synkat.

REQUIREMENTS/HIGH-LEVEL/SAFETY-MONITORING.MD

```
# HLR-SAFE-001: Engine overheat protection
**ASIL:** D
**Rationale:** Förhindra katastrofal motorskada

Systemet ska detektera kylvätsketemperatur > 115 °C
inom 100 ms och initiera nödstopp.

Spårbarhet: → LLR-1.4-temp-monitor, LLR-3.2-shutdown
```

MANUS

Här är ett verkligt HLR. Notera vad det inte säger.

Det säger ingenting om vilken ADC-kanal vi mäter på. Inget om vilket protokoll som ska kommunicera nödstoppet. Inget om hur vi implementerar timern. Inget om vilken plattform.

Det säger bara *vad* — beteendet — och *varför* — säkerhetsskälet. ASIL-nivån D säger oss att detta är högsta säkerhetsklassen. Spårbarhetslänkarna pekar på de LLR:er som tar hand om detaljerna.

En domänexpert kan skriva detta. En jurist kan läsa det. En revisor från TÜV kan auditera det. Ingen av dem behöver kunna C.

Det är hela poängen.

Shot list — Low-Level Requirements.

TAKEAWAYS

- Strukturerad YAML
- Konflikter detekteras automatiskt
- Ändra → regenerera → klart

SÖKVÄGAR

- requirements/low-level/*.yaml
- scripts/reqgen.py
- scripts/conflict_handler.py

MANUS

Om HLR är manuset är LLR shot list — den detaljerade listan över exakt vilka tagningar som ska göras.

LLR är YAML. Strukturerad, maskinläsbar, men fortfarande mänsklig — ingen kod, ingen pseudokod. Bara fakta: den här ADC-kanalen, det här tröskelvärdet, den här toleransen, det här protokollet, den här hertzfrekvensen.

Varje LLR har en `parent` som pekar tillbaka till sitt HLR. Det betyder att spårbarheten är *inneboende* — du kan inte skriva ett LLR utan att deklarerat vilket HLR det realiserar.

Det andra som händer är konfliktdetektering. Modulen `conflict_handler.py` läser alla LLR:er och letar efter motstridigheter — samma signal med olika gränser, två LLR:er som båda hävdar äganderätt över samma utgång, dödläsningar i schemaläggningen. När en konflikt hittas i STRICT-läge stannar bygget. Du kan inte producera en binär från motstridiga krav.

REQUIREMENTS/LOW-LEVEL/LLR-1.1-RPM-SAMPLING.YAML

```
id: LLR-1.1-rpm-sampling
parent: HLR-CTRL-001
input:
  type: adc
  channel: 0
  stale_us: 5000
redundancy:
  pair_with: LLR-1.2-rpm-spi
  vote: median
  tolerance: 50
output:
  protocol: j1939
  pgn: 0xFEf1
  rate_hz: 10
```

MANUS

En tagning. Tolv rader.

Notera redundansblocket. Det här LLR:et säger: vi har en RPM-sensor på ADC-kanal noll. Den ska paras ihop med LLR-1.2 som är samma RPM mätt över SPI. Median-voting används för att slå ihop dem. Om de skiljer sig mer än 50 RPM markeras inkonsistens.

Och utgången: J1939-protokollet, PGN 0xFEf1, tio gånger per sekund.

Tolv rader YAML genererar — vänta tills nästa slide — något i storleksordningen 387 rader produktionsfärdig C-kod. ISR-handlers, lock-free ringbuffer-pushes, validering, voter-logik, deadline-scheduling, J1939-frame-encoding, allt. Hand på hjärtat: hur många buggar tror ni det blir när en människa skriver de raderna?

Genererad kod har deterministiska buggar. Hittas en, fixas den i generatoren, försvinner från alla LLR:er.

REEL VII · THE STORYBOARD

Device Tree är vår storyboard.

TAKEAWAYS

- Komplet, deterministisk beskrivning
- Genereras från krav
- Läses även av Zephyr-kärnan

SÖKVÄGAR

- `${BUILD}/app.dts`
- `samples/automotive/app.dts`
- `dts/bindings/`

MANUS

Mellan manuset och inspelningen ligger storyboarden. För filmare är det en serie tecknade rutor, en per scen, som visar exakt vad kameran ska se. Det är vad regissören delar med ljusmästaren och scenografen och kostymdesignern.

I vårt system är storyboarden Device Tree. DTS är ett gammalt format från Linux-världen som beskriver hårdvara hierarkiskt. Zephyr använder det för allt — vilka pinnar är vad, vilka bussar finns, hur är de kopplade.

Vi har utökat det. Vår DTS beskriver inte bara hårdvaran utan hela databehandlingsdiagrammet: ingångar, mid-level-validatorer, voters, range-checkers, utgångar, deadlines.

Det fina är att DTS:en *redan läses* av Zephyrs egen kärna och drivers. Vi piggybackar på ett standardformat som varje embedded-utvecklare redan kan. Inget nytt språk att lära sig. Och det är genererat — inget vi handskriver.

APP.DTS – AUTOGENERERAD FRÅN KRAV

```
rpm_adc: lq-hw-adc-input@0 {
    compatible = "lq,hw-adc-input";
    io-channels = <&adc0 0>;
    stale-us    = <5000>;
};

rpm_merge: lq-mid-merge@0 {
    compatible    = "lq,mid-merge";
    sources       = <&rpm_adc &rpm_spi>;
    voting-method = "median";
    tolerance     = <50>;
};
```

MANUS

Här är delen av DTS:en som motsvarar vårt LLR från förra sliden.

Två noder: en ADC-ingång och en merge-nod som tar två sourcer och voterar. Allt med `compatible`-strängar som matchar våra Zephyr-bindings i `dtb/bindings/`.

Den intressanta detaljen: Zephyrs *egen* DTS-kompilator validerar den här filen mot bindingsen. Det betyder att om generatoren genererar något som inte är välformat — fel typ på ett fält, saknad property — så fångas det redan av Zephyr, innan vår C-kodgenerator ens körs.

Vi får alltså validering i två lager: vår egen `reggen.py validate` på krav-nivå, och Zephyrs DTS-validering på arkitektur-nivå. Två oberoende grindar. Båda måste passera innan en enda C-rad genereras.

Filmteamet.

MODULER

- core · config · platform
- hil · uds
- Samma manus, olika kameror

PLATTFORMAR

- scripts/generators/platforms/zephyr.py
- stm32.py · esp32.py
- nrf52.py · samd.py · baremetal.py

MANUS

Bakom varje regissör finns ett team. I filmen kallar vi DP — director of photography, fotograf — och så finns det specialister för varje aspekt: ljus, kamera, ljud, effekter.

I vår kodgenerator är teamet modulärt uppdelat. `core.py` är produktionsledaren — den orkestrerar. `config.py` läser DTS:en och bygger den interna modellen. `platform.py` väljer vilken kamera som ska användas. `hil.py` är vår testkamera. `uds.py` hanterar diagnostik.

Och i `platforms/`-mappen ligger varje "kamera": `zephyr.py`, `stm32.py`, `esp32.py`, `nrf52.py`, `samd.py`, `baremetal.py`. Samma manus — samma DTS — kan filmas med vilken som helst av dem. Du byter kamera med en kommandoradflagga.

Det är arkitekturen som gör det möjligt att portera till en ny plattform genom att skriva en ny modul, inte genom att modifiera generatorns kärna. Open-closed-principen i praktiken.

Scenografin — Input → Engine → Output.

TAKEAWAYS

- Tunna lager mot hårdvara
- Pure processing-engine
- Pollning > ISR där det går

SÖKVÄGAR

- `src/drivers/lq_engine.c`
- `src/drivers/lq_hw_input.c`
- `include/lq_engine.h · lq_event.h`

MANUS

Bakom kulisserna måste alla scener spelas in i en scenografi. Och i vår kodbas är scenografin oerhört enkel — och det är det som gör den säker.

Tre lager. Inget mer.

Lager ett: ett tunt, RTOS-medvetet hårdvarulager. ISR:er och pollning. Det enda jobbet är att ta in data från fysiken och skuffa in den i en lock-free ringbuffer. Inga affärsregler här.

Lager två: en *ren* processing-engine. Pure. Inga RTOS-anrop, ingen dynamisk minnesallokering, ingen hårdvarukännedom. Bara logik som äter events från ringbuffern och spottar ut events till utgångskön. Det är detta lager som är unit-testbart, formellt verifierbart, kompillerbart till ren x86 för testning.

Lager tre: utgångsdrivers — också tunna, hårdvarunära.

En subtilitet: vi pollar där vi kan. GPIO, ADC, SPI, I2C — det är *deterministiskt* timad sampling, inte interrupt-driven. Endast asynkrona protokoll som CAN och UART använder ISR, eftersom meddelanden där kommer när de kommer. Determinism är inte en känsla, det är en arkitekturbeslut.

ARKITEKTUR – TVÅ TUNNA HW-LAGER, REN KÄRNA

```

ISR / Polling
  | lq_hw_push()
Lock-free ringbuffer
  | lq_hw_pop()
Mid-level vtables    (PURE)
  |
ENGINE STEP          (deterministic)
  |
Output dispatch      (bounded)
  |
CAN · J1939 · CANopen · GPIO · PWM · DAC · SPI · I2C · UART · Modbus

```

MANUS

Här är samma sak som ett ASCII-diagram.

Notera ordet PURE. Mellan-lagret — där voters, validators och range-checkers bor — är ren kod. Inga locks, inga mutexes, inga RTOS-anrop. Det betyder att det går att köra exakt samma kod på en utvecklingsmaskin som i firmwaren. Vi kompilerar samma kärna med GCC på Linux för unit-tester, och med arm-none-eabi-gcc för målsystemet. Identisk bytkod-logik.

ENGINE STEP är det centrala. Ett anrop per cykel. Ingen multitasking inuti, ingen icke-determinism. Tar in events, spottar ut events. Bounded execution time — vi kan räkna ut den värsta tänkbara körtiden statistiskt.

Output dispatch är en simpel for-loop. Inget jobbflöde, inga köer. När engine-step är klar, dispatchar vi alla utgångsevent som är schemalagda för den cykeln. Det är allt.

Det är inte raketforskning. Det är gammal aerospace-arkitektur, DO-178C-stil. Men *det är genererat från krav*.

Stop worrying. Love the generator.

TAKEAWAYS

- Ingen AI vid generering
- Bit-identisk output
- Tool qualification: ISO 26262-8

BEVIS

- 444 tester
- 92–100% kritisk täckning
- gitleaks · ruff (hooks)

MANUS

Stanley Kubricks "Dr. Strangelove or: How I Learned to Stop Worrying and Love the Bomb" handlar om att lära sig leva med teknologi som är skrämmande men oundviklig. Idag måste vi lära oss något liknande om kodgenerering.

Den första frågan jag får är alltid: "Använder ni AI för att skriva säkerhetskritisk kod?" Svaret är *nej*. AI hjälpte till att bygga *generatorn*. Generatorn själv är ren, deterministisk Python. Samma `requirements/` och samma version av generatorn ger bit-identisk `lq_generated.c`. Du kan checka in genererad kod i Git och se den som data, inte som artefakt.

Det här passar perfekt in i ISO 26262-8: Tool Qualification. Du behöver inte certifiera *koden* den genererar — du certifierar *generatorn*, en gång, och sedan kan vilken kod den genererar som helst räknas som korrekt så länge inputen är korrekt.

Vi har 444 automatiserade tester på generatorn själv. 92 till 100% täckning av kritiska paths. Pre-commit-hooks med gitleaks och ruff. Generatorn behandlas som vad den är: säkerhetskritisk mjukvara.

V-modellen sluts.

VÄNSTER SIDA — SW

- Unit-tester per LLR
- GoogleTest · CTest
- Mock av HW-gränssnitt

HÖGER SIDA — HW

- tests/hil/
- scripts/hil_test_gen.py
- generate_comprehensive_hil_tests.py

MANUS

Före premiären har varje stor film en testvisning. Publik bjuds in, reagerar, och regissören klipper om baserat på reaktioner.

För oss är det dubbla testvisningar — V-modellens båda sidor.

Vänster sida, mjukvaruverifiering: unit-tester, ett per LLR, autogenererade. GoogleTest under CTest. Mockade hårdvarugränssnitt så vi kan köra på vilken laptop som helst. När du ändrar ett LLR, regenereras dess test, och CI fångar regression.

Höger sida, hårdvaruverifiering: HIL-scenarier, också autogenererade från samma LLR. Skickas till en testtrigg med riktig STM32, riktig CAN-buss, riktig sensor. Vi injicerar 120 °C på temperatursensorn — verifierar att shutdown sker inom 100 ms.

Det här är V-modellens heliga gral: *bevisad korrespondens mellan krav och verifiering*. Inte hoppat på, utan inneboende. Samma rad i samma fil genererar både implementation och dess test. Du kan inte ändra ena utan den andra. Och du kan inte få ut en grön CI utan båda.

Action — kameran rullar.

TAKEAWAYS

- Klipp manuset
- west build (Action)
- west flash + ctest (Premiär)

SÖKVÄGAR

- scripts/build_firmware.sh
- scripts/west_build_with_prjconf.sh
- build/ · Testing/

MANUS

Nu sätter vi ihop allt. Ett verkligt arbetsflöde.

Steg ett: någon — en säkerhetsanalytiker, en domänexpert — öppnar ett LLR och justerar ett toleransvärde. En liten textändring. Commit, push, pull request.

Steg två: jag, eller CI, kör `west build`. Och nu händer magin. CMake konfigurerar. Vår `RequirementsDriven.cmake` aktiveras. `reqgen.py` validerar i STRICT-läge. Genererar DTS. `dts_gen.py` genererar C-koden. Ninja kompilerar. Linker länkar. Ut kommer en `zephyr.elf`.

Steg tre: `west flash` skickar binären till hårdvaran. `ctest` kör hela testsviten. 444 av 444 grönt. Och vi har en uppdaterad produkt.

Mellan steg ett och steg tre har ingen människa skrivit en rad C-kod. Ingen har uppdaterat en spårbarhetsmatris. Ingen har skrivit ett test. Ingen har uppdaterat dokumentationen. Allt detta är *biprodukter* av byggprocessen.

EN TAGNING – KRAV → FLASHBAR BINÄR

```
$ vim requirements/low-level/llr-1.1-rpm-sampling.yaml

$ west build -b nucleo_f429zi samples/automotive
  → reggen.py validate (STRICT)
  → reggen.py generate-dts → build/prereq/app.dts
  → dts_gen.py             → lq_generated.[ch]
  → ninja                  → zephyr.elf

$ west flash
$ ctest --test-dir build          # 444 / 444
```

MANUS

Här är hela kommandorad-sekvensen.

Det första kommandot — `vim` — är den enda mänskliga insatsen. Resten är automatik. Lägg märke till hur `west build` triggar en kedja av Python-skript: `validate`, `generate-dts`, `dts_gen`. Och därefter Ninja som kompilerar.

Det är värt att stanna upp på `STRICT` här. Det är rekommenderat läge för CI och för release-byggen. Det betyder att en varning från konflikt-handlern stoppar bygget. En mjukare `NORMAL` kan vara rimlig under utveckling när du vet att du har konflikter du jobbar på.

Och `444 / 444` på sista raden — det är inte en marknadsföringssiffra. Det är det verkliga antalet tester som körs på varje commit. Misslyckas en — bygget rött. Misslyckas inga — färdigt att flasha.

Det här är vad "smidig och säker" betyder i praktiken: en commit, en kommando, en deterministisk binär.

CI är distributören.

TAKEAWAYS

- Samma kommando som dev
- Ingen "works on my machine"
- Coverage-badge alltid synkad

SÖKVÄGAR

- `.github/workflows/`
- `docs/CI_HIL_SETUP.md`
- pre-commit: `gitleaks` + `ruff`

MANUS

När filmen är klipplagd och godkänd är distributören som tar den vidare till biograferna. För oss är distributören CI-pipelinen.

Det viktiga med vår CI är att den inte gör något *speciellt*. Den kör exakt samma kommandon som du och jag kör lokalt: `west init`, `west update`, `west build`, `ctest`. Inga särskilda CI-mode-flaggor som drifter över tid. Det betyder ingen "works on my machine" — om bygget passerar lokalt, passerar det i CI. Om CI misslyckas, kan du reproducera felet på din laptop på minuter.

Coverage-badgen i README:n regenereras automatiskt vid varje merge. Den är aldrig osynkad med koden. Du har inga gamla siffror som ljuger.

På ingångssidan har vi pre-commit-hooks. `gitleaks` förhindrar att hemligheter checkas in. `ruff` linter och formaterar Python-koden i generatoren. Hookar är inte valbara — det är en organisationsregel hos oss, eftersom säkerheten på *verktyget* är säkerheten på *produkten*.

Vad det betyder.

TAKEAWAYS

- Designa, inte översätta
- "What if" blir billigt
- Spårbarhet som biprodukt

KONSEKvens

- Ny plattform → samma manus
- 10–15 h byggtid med AI-stöd
- Ekonomisk barriär: borta

MANUS

Slutet närmar sig. Vad har vi sett?

Det första: ingenjörer slutar översätta och börjar designa. Tiden som tidigare gick åt till att skriva ISR-handlers och voter-logik fri från fel går nu åt till att verkligen *fundera* över systembeteende. Det är ett yrkesskifte — från hantverkare till arkitekt.

Det andra: "What if" blir billigt. Vad händer om vi byter från ASIL B till ASIL D? Regenerera och kör testerna. Vad händer om vi byter från CAN till J1939? Ändra protokoll-fältet i LLR:en. Vad händer om vi vill stödja en fjärde plattform? Skriv en ny generator-modul. Iteration som kostade veckor kostar nu minuter.

Det tredje, och kanske viktigaste: spårbarhet är en biprodukt. Inte ett dokumentationsprojekt som körs i sista minuten innan TÜV-inspektion. Det är inneboende i hur vi bygger.

Och slutligen: hela det här verktyget byggdes på 10 till 15 timmar med AI-assistans. Den ekonomiska barriären — som tidigare var "200 timmar och vi har inte budget" — är borta. Det är inte AI som skriver din säkerhetskritiska kod. Det är AI som gjorde det möjligt att bygga *verktyget* som genererar den.

"Its origin and purpose, still a total mystery."

— Monoliten i 2001. Vår är generatorn — liten, svart, deterministisk. Och fullt revisionsbar.

MANUS

Och så åter till 2001.

I Kubricks film dyker monoliten upp på platser där mänsklig evolution tar ett språng. Den är liten, svart, deterministiskt rektangulär — sidorna i förhållandet 1 till 4 till 9, kvadrater på de tre första heltalen. Den är obegriplig för dem som finner den, men dess effekt är obestridlig.

Vår monolit är kodgeneratorn. Liten — några tusen rader Python. Svart — i den mening att den är opaque för användaren, du behöver inte förstå dess insida för att använda den. Deterministisk — samma input, samma output, varje gång.

Men till skillnad från Kubricks monolit är vår *fullt revisionsbar*. Du kan öppna den, läsa varje rad, köra varje test, granska varje commit i dess historik. Den är ett mysterium endast för den som väljer att inte titta.

Och det är skillnaden mellan magi och teknik. Båda imponerar. Bara den ena bär du in i en certifieringsaudit.

The End.

Frågor är välkomna · git clone är ännu varmare

ENTRY POINTS

- `cmake/RequirementsDriven.cmake`
- `scripts/reqgen.py`
- `west.yml`

MANUS & SCEN

- `requirements/high-level/`
- `requirements/low-level/`
- `samples/automotive/`

MANUS

Tack för uppmärksamheten.

Fyra platser i repot är värda en personlig titt om ni vill gå vidare:

För det första, `cmake/RequirementsDriven.cmake` — själva regissörsstolen, det enda CMake-anropet ni behöver veta om.

För det andra, `scripts/reqgen.py` — kärnan i kravbehandlingen och konfliktdetekteringen.

För det tredje, `west.yml` — manifestet som binder ihop hela studion.

Och för det fjärde, `samples/automotive/` — ett komplett, körbart exempel: en motorövervakare med dubbla RPM-givare, median-voting och J1939-utgång. Bygg och kör det själva — det är det bästa sättet att förstå vad jag har pratat om.

Frågor?

Och som sagt: en `git clone` är välkomnare än en fråga, för då kan ni ge feedback också.